

Week 05

함수·모듈·테스트 가능한 코드

이영호 교수 | 국립목포대학교 컴퓨터학부

오늘의 목표와 산출물

- 핵심 개념: def, 매개변수, return, 기본값, 모듈, docstring, assert
- 실습: 재사용 가능한 성적 계산 함수
- 산출물: 입력·처리·출력을 분리한 함수 모음

Colab 실습 지도

단계	슬라이드	노트북	학생 활동
개념 그림	5	흐름 안내	코드가 처리되는 순서 설명
Level 1 따라하기	6~8	첫 코드 셀	예상 → 실행 → 한 곳 변경
Level 2 바꾸기	9~10	핵심 코드 셀	값·조건 두 곳 변경
Level 3 적용하기	11~12	도전 코드 셀	정상·경계 사례 테스트
Level 4 확장하기	13~15	확장 코드 셀	기능 설계·AI 기록

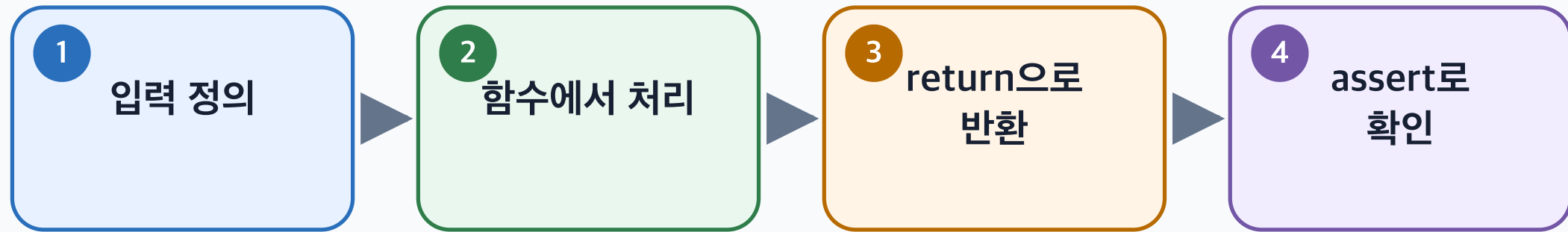
이번 주 Colab 열기

핵심 개념 연결

- def
- 매개변수
- return
- 기본값
- 모듈
- docstring
- assert

개념 순서도

테스트 가능한 함수 만들기



입력과 출력이 분명한 작은 함수는 재사용과 테스트가 쉽습니다.

AI 활용 Python 프로그래밍 · Week 05

Level 1 — 먼저 예측하기

가장 쉬운 코드부터 시작합니다

1. 코드를 아직 실행하지 않습니다.
2. 각 줄의 역할과 출력 결과를 예상합니다.
3. 예상 내용을 옆 학생에게 한 문장으로 설명합니다.

Level 1 — 따라하기

```
def greet(name):  
    return f"안녕하세요, {name}님!"  
  
message = greet("민수")  
print(message)
```

실행 전 질문: 어떤 값이 출력될까요?

Level 1 — 한 곳만 바꾸기

1. 숫자 또는 문자열 한 곳을 찾습니다.
2. 값을 바꾸면 어떤 출력이 나올지 먼저 적습니다.
3. Colab에서 실행하여 예상과 비교합니다.

설명 문장: “___을 ___로 바꾸었기 때문에 결과가 ___로 달라졌다.”

Level 2 — 핵심 코드 읽기

```
def letter_grade(score: float) -> str:
    """0~100점 점수를 문자 등급으로 변환한다."""
    if not 0 <= score <= 100:
        raise ValueError("점수는 0~100 범위여야 합니다.")
    if score >= 90:
        return "A"
    if score >= 80:
        return "B"
    if score >= 70:
        return "C"
    return "F"

assert letter_grade(95) == "A"
assert letter_grade(70) == "C"
print(letter_grade(84))
```

코드의 입력 → 처리 → 출력 부분을 각각 표시하세요.

Level 2 — 두 곳 바꾸기

- 입력값.조건.데이터 중 한 곳을 변경합니다.
- 계산.함수.집계 방식 중 한 곳을 변경합니다.
- 변경 전후 결과를 표로 기록합니다.

구분	변경 전	변경 후	달라진 이유
실행 결과			

Level 3 — 문제에 적용하기

```
def summarize_scores(scores):  
    if not scores:  
        return {"count": 0, "average": None}  
    return {"count": len(scores), "average": round(sum(scores) / len(scores), 1)}  
  
assert summarize_scores([])["average"] is None  
print(summarize_scores([80, 90, 100]))
```

노트북의 전체 코드를 실행하고 요구사항과 연결하세요.

Level 3 — 테스트로 확인하기

1. 정상 입력과 예상 출력을 작성합니다.
2. 경계값 또는 오류 가능 입력을 하나 추가합니다.
3. 실제 결과와 비교합니다.
4. 실패하면 오류 메시지와 수정 이유를 기록합니다.

“실행됨”과 “정확함”을 따로 확인합니다.

Level 4 — AI와 기능 확장

아래 Python 코드에 [추가 기능]을 넣고 싶습니다.
먼저 변경할 위치와 이유만 설명해 주세요.
전체 코드를 바로 작성하지 말고,
정상 사례와 경계 사례 테스트를 각각 제안해 주세요.

- AI 제안 중 채택·거절한 부분을 구분합니다.
- 직접 수정한 줄과 테스트 결과를 기록합니다.

실습 완료 점검

- [] Level 1: 예상 → 실행 → 한 곳 변경
- [] Level 2: 두 곳 변경과 결과 설명
- [] Level 3: 정상·경계 사례 테스트
- [] Level 4: 확장 기능 또는 설계 메모
- [] AI Prompt·채택·수정·검증 기록

오늘의 산출물과 마무리

입력·처리·출력을 분리한 함수 모음

- 노트북이 위에서 아래로 실행되는가?
- 코드 한 부분을 말로 설명할 수 있는가?
- AI 제안을 정상·경계 사례로 검증했는가?
- 직접 바꾼 코드와 이유가 기록되어 있는가?